

ADR 001: Arquitectura de Gráficos Dependientes (Monolítica)

Observatorio Técnico Catastral

ADR (Architecture Decision Record)

- **Documento:** ADR001
- **Título:** Arquitectura de Gráficos Dependientes (Monolítica)
- **Autor:** Daniel Santiago Cristancho Moreno
- **Fecha:** Octubre 2025
- **Estado:** Aprobado
- **Versión:** 1.0

Índice

1. Resumen Ejecutivo
2. Metadatos del Documento
3. Contexto y Problema
4. Declaración de Decisión
5. Alternativas Consideradas
6. Justificación
7. Consecuencias
8. Implementación
9. Comparación con Arquitectura Independiente
10. Decisiones Relacionadas
11. Referencias

1. Resumen Ejecutivo

Se ha implementado una **arquitectura monolítica de gráficos dependientes** para el Observatorio Técnico Catastral. Esta aproximación consolida todos los 15 componentes de visualización en un archivo HTML único e integrado, priorizando simplicidad de desarrollo y despliegue sobre flexibilidad de configuración.

Limitación fundamental: Cambiar la posición y orden de los gráficos requiere conocimientos fundamentales en programación, ya que la estructura está definida directamente en el código fuente. Sin embargo, técnicamente es factible realizar estas modificaciones para usuarios con experiencia técnica.

Esta decisión arquitectural ha sido validada y aprobada como la solución óptima para los requerimientos específicos del proyecto, considerando los recursos disponibles y las necesidades operacionales de la organización.

2. Metadatos del Documento

Campo	Valor
ID del ADR	ADR001
Título	Arquitectura de Gráficos Dependientes (Monolítica)
Estado	Aprobado
Fecha de Decisión	Octubre 2025

Campo	Valor
Área de Impacto	Arquitectura Frontend, Despliegue, Mantenimiento
Partes Interesadas	Equipo de Desarrollo, Administrador Web, Área de Tecnología
Revisor Técnico	Arquitecto de Software Senior
Aprobado por	Director de Tecnología

3. Contexto y Problema

3.1 Situación de Decisión Arquitectural

El equipo de desarrollo del Observatorio Técnico Catastral evaluó dos enfoques arquitecturales fundamentalmente diferentes:

- **Gráficos Independientes:** Componentes modulares y separados
- **Gráficos Dependientes (Implementado):** Estructura monolítica integrada

3.2 Requisitos del Sistema

Funcionalidades Principales:

- 15 tipos de gráficos diferentes (barras, líneas, circular, mapas, radar)
- Visualización de datos inmobiliarios de Bogotá por localidades, estratos, áreas, etc.
- Exportación de datos a Excel para análisis adicional
- Diseño adaptativo para múltiples dispositivos
- Actualización periódica de información

Consideraciones Organizacionales:

- **Personal disponible:** Equipo de desarrollo pequeño
- **Recursos técnicos:** Limitados para mantenimiento complejo
- **Cronograma:** Necesidad de despliegue rápido
- **Flexibilidad futura:** Incierta en términos de reorganización de contenido

3.3 Problema de Flexibilidad vs. Simplicidad

Dilema Fundamental:

¿Priorizar flexibilidad de configuración o simplicidad de desarrollo/mantenimiento?

- **Flexibilidad:** Capacidad de reorganizar gráficos sin modificar código
- **Simplicidad:** Menor complejidad técnica, despliegue más directo
- **Compromiso inevitable:** No es posible maximizar ambas características simultáneamente

4. Declaración de Decisión

Decisión implementada: Arquitectura monolítica de gráficos dependientes donde todos los 15 componentes de visualización se integran en un archivo HTML único y cohesivo.

4.1 Estructura Monolítica

- **Un archivo HTML principal:** observatorio-tecnico-catastral.html
- **Gráficos integrados:** Todos los componentes en secciones del mismo documento
- **Estilos unificados:** CSS global para consistencia visual
- **JavaScript centralizado:** Lógica de gráficos en el mismo archivo o archivos mínimos

4.2 Limitación Aceptada

Modificación de diseño requiere programación: Cambiar posición, orden o configuración de gráficos necesita editar código fuente, requiriendo conocimientos técnicos de HTML, CSS y JavaScript.

4.3 Beneficio Principal

Simplicidad operacional máxima: Un solo archivo para despliegue, respaldo, control de versiones y mantenimiento básico del portal completo.

5. Alternativas Consideradas

5.1 Alternativa 1: Gráficos Completamente Independientes

Descripción: Cada gráfico como archivo HTML separado con sus propios recursos.

Ventajas:

- **Máxima flexibilidad:** El administrador web puede reorganizar sin limitaciones
- **Mantenimiento granular:** Actualizar gráficos individualmente
- **Escalabilidad:** Fácil agregar nuevos gráficos
- **Separación de responsabilidades:** Cada componente autónomo

Desventajas:

- **Complejidad de despliegue:** 15+ archivos para gestionar
- **Sobrecarga de recursos:** Duplicación de librerías y estilos
- **Coordinación compleja:** Consistencia entre componentes independientes
- **Mayor superficie de fallo:** Más puntos de potencial error

Razón de rechazo: La complejidad operacional supera los beneficios de flexibilidad para un equipo de recursos limitados.

5.2 Alternativa 2: Híbrida Modular

Descripción: Gráficos agrupados en módulos temáticos (3-4 archivos HTML).

Ventajas:

- **Equilibrio complejidad-flexibilidad:** Menos archivos que completamente independiente
- **Agrupación lógica:** Gráficos relacionados juntos
- **Flexibilidad parcial:** Reorganización dentro de módulos
- **Mantenimiento moderado:** Menos complejo que 15 archivos separados

Desventajas:

- **Flexibilidad limitada:** No permite reorganización total
- **Complejidad residual:** Aún requiere gestión de múltiples archivos
- **Decisiones arbitrarias:** Qué gráficos agrupar juntos
- **No resuelve problema principal:** Sigue requiriendo conocimientos técnicos

Razón de rechazo: Mantiene complejidad sin ofrecer beneficios decisivos sobre alternativas puras.

5.3 Alternativa 3: CMS o Framework Dinámico

Descripción: Utilizar un sistema de gestión de contenido para configuración visual.

Ventajas:

- **Flexibilidad total:** Configuración sin conocimientos de código
- **Interfaz gráfica:** Arrastrar y soltar para reorganizar componentes
- **Roles de usuario:** Diferentes niveles de acceso y permisos

- **Funcionalidad avanzada:** Programación de actualizaciones, versiones, etc.

Desventajas:

- **Complejidad técnica extrema:** Requiere configuración de base de datos, servidor, framework
- **Costo de desarrollo:** Semanas/meses de desarrollo adicional
- **Dependencias externas:** Frameworks, alojamiento especializado
- **Excesivo evidente:** Desproporcionado para 15 gráficos relativamente estáticos
- **Curva de aprendizaje:** El personal debe aprender nuevo sistema

Razón de rechazo: La complejidad y costo no se justifican para los requerimientos específicos del proyecto.

5.4 Alternativa 4: Sistema de Plantillas Básico

Descripción: Sistema de plantillas simple con configuración JSON externa.

Ventajas:

- **Configuración externa:** Cambios sin modificar HTML
- **Simpleidad relativa:** Menos complejo que CMS completo
- **Flexibilidad estructurada:** Reorganización con archivos de configuración
- **Mantenimiento centralizado:** Una plantilla, múltiples configuraciones

Desventajas:

- **Desarrollo adicional:** Requiere crear sistema de plantillas desde cero
- **Complejidad de configuración:** JSON complejo para diseños avanzados
- **Depuración compleja:** Los errores pueden venir de plantilla o configuración
- **Conocimientos técnicos:** Igualmente requiere entender JSON y estructuras

Razón de rechazo: Mantiene la barrera técnica mientras introduce complejidad de desarrollo adicional.

6. Justificación

6.1 Factores Decisivos

1. Simplicidad de Desarrollo Máxima

La arquitectura monolítica permite desarrollo directo:

- **Un archivo principal:** Todo el código en ubicación conocida
- **Sin coordinación entre archivos:** No hay dependencias externas complejas
- **Depuración simplificada:** Todos los componentes en el mismo contexto
- **Pruebas directas:** Cargar un archivo y verificar funcionamiento completo

2. Despliegue y Mantenimiento Mínimos

Aspecto	Monolítica	Independientes
Archivos a desplegar	1	15+
Tiempo de despliegue	2 minutos	30 minutos
Complejidad de respaldo	Mínima	Alta
Puntos de fallo	1	15+
Sincronización requerida	No	Sí

3. Costo de Flexibilidad vs. Beneficio Real

Análisis de frecuencia de cambios de diseño:

- **Reorganización mayor:** ¿Con qué frecuencia se necesita realmente?
- **Cambios menores:** Pueden ser implementados por desarrollador
- **Costo de flexibilidad:** Complejidad permanente vs. beneficio ocasional

4. Recursos del Equipo

- **Equipo pequeño:** Menos sobrecarga de gestión con arquitectura simple
- **Conocimientos técnicos disponibles:** El equipo puede manejar modificaciones cuando sean necesarias
- **Cronograma de proyecto:** La arquitectura simple permite entrega más rápida

6.2 Análisis Costo-Beneficio

Criterio	Independientes	Híbrida	CMS/Framework	Sistema Plantillas	Monolítica
Flexibilidad sin código	✓ Máxima	⚠ Parcial	✓ Total	⚠ Con JSON	✗ Requiere código
Simplicidad desarrollo	✗ Compleja	⚠ Media	✗ Muy compleja	⚠ Media	✓ Muy simple
Simplicidad despliegue	✗ Complejo	⚠ Medio	✗ Muy complejo	⚠ Medio	✓ Muy simple
Costo desarrollo	⚠ Medio	⚠ Medio	✗ Muy alto	⚠ Alto	✓ Muy bajo
Mantenimiento	✗ Complejo	⚠ Medio	✗ Muy complejo	⚠ Medio	✓ Simple
Tiempo de implementación	⚠ Medio	⚠ Medio	✗ Lento	✗ Lento	✓ Muy rápido

6.3 Aceptación Consciente de Compromisos

Compromiso Principal Aceptado:

Flexibilidad ↔ Simplicidad

Decisión: Priorizar simplicidad operacional sobre flexibilidad de configuración, basado en:

- Frecuencia real de cambios de diseño (proyectada como baja)
- Disponibilidad de conocimientos técnicos en el equipo para cambios ocasionales
- Valor de simplicidad para mantenimiento día a día vs. valor de flexibilidad para cambios ocasionales

7. Consecuencias

7.1 Consecuencias Positivas

Para el Equipo de Desarrollo:

- **Desarrollo acelerado:** Un solo archivo para crear y mantener
- **Pruebas simplificadas:** Cargar una página y verificar todos los gráficos
- **Depuración directa:** Todos los elementos en el mismo contexto
- **Control de versiones simple:** Historial claro con un archivo principal

Para el Área de Tecnología:

- **Despliegue trivial:** Un archivo al servidor, proceso completo
- **Respaldo simple:** Un archivo para respaldar todo el observatorio
- **Resolución de problemas directa:** Un lugar para buscar problemas
- **Recursos mínimos:** Sin infraestructura compleja necesaria

Para la Organización:

- **Tiempo de comercialización mínimo:** Portal funcionando en semanas, no meses
- **Costo de desarrollo reducido:** Sin sobrecarga arquitectural compleja
- **Mantenimiento predecible:** Proceso simple y bien definido
- **Riesgo técnico bajo:** Menos componentes móviles = menos puntos de fallo

7.2 Consecuencias Negativas y Mitigaciones

1. Flexibilidad de Diseño Limitada

- **Impacto:** Cambios de posición/orden de gráficos requieren modificar código HTML
- **Evaluación del impacto real:**
 - Frecuencia proyectada: ¿Cada cuánto se reorganiza realmente un tablero de control?
 - Disponibilidad de recursos: ¿El equipo puede hacer estos cambios cuando son necesarios?
 - Alternativas: ¿CSS y JavaScript pueden proporcionar cierta flexibilidad sin cambios estructurales?

Mitigación:

- **Documentación clara:** Guías paso a paso para modificaciones comunes
- **Estructura HTML semántica:** Secciones claramente identificadas para facilitar cambios
- **CSS flexible:** Usar CSS Grid/Flexbox para facilitar reorganización con cambios mínimos
- **Pruebas automatizadas:** Scripts para verificar que cambios no rompan funcionalidad

2. Acoplamiento de Componentes

- **Impacto:** Problemas en un gráfico pueden afectar otros o todo el sistema

Mitigación:

- **Manejo de errores robusto:** Cada gráfico maneja sus errores independientemente
- **Bloques try-catch:** Aislar fallos de gráficos individuales
- **Degradación elegante:** Mostrar mensaje de error sin romper otros gráficos
- **Pruebas exhaustivas:** Verificar funcionamiento con datos faltantes o erróneos

3. Escalabilidad de Código

- **Impacto:** El archivo puede volverse grande y complejo con el tiempo

Mitigación:

- **Organización clara:** Secciones bien definidas y comentadas
- **Modularización interna:** JavaScript organizado en funciones específicas por gráfico
- **Separación de responsabilidades:** Datos, lógica y presentación claramente separados
- **División de código cuando sea necesario:** Si el archivo crece demasiado, dividir en módulos

4. Mantenimiento de Consistencia

- **Impacto:** Cambios globales requieren editar múltiples secciones del mismo archivo

Mitigación:

- **Estilos centralizados:** CSS global para evitar duplicación
- **Plantillas JavaScript:** Funciones reutilizables para gráficos similares
- **Variables globales:** Configuración centralizada para colores, fuentes, etc.
- **Convenciones estrictas:** Patrones de código consistentes en todo el archivo

8. Implementación

8.1 Estructura Monolítica Propuesta

Archivo Principal: observatorio-tecnico-catastral.html

```
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Observatorio Técnico Catastral</title>

  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://cdn.jsdelivr.net/npm/echarts@5.4.3/dist/echarts.min.js"></script>
  <script src="https://cdn.jsdelivr.net/npm/xlsx@0.18.5/dist/xlsx.full.min.js"></script>

  <style>
    .chart-container {
      width: 100%;
      height: 400px;
      min-height: 300px;
    }
    .chart-section {
      margin-bottom: 2rem;
    }
    /* Estilos globales adicionales */
  </style>
</head>
<body class="bg-gray-100">

  <header class="bg-blue-600 text-white p-6">
    <h1>Observatorio Técnico Catastral</h1>
  </header>

  <main class="container mx-auto px-4 py-8">

    <section class="chart-section bg-white rounded-lg shadow-lg p-6">
      <div>
        <h2>Evolución de Transacciones</h2>
        <button onclick="exportLineasData()"
          class="bg-yellow-400 hover:bg-yellow-500 px-4 py-2 rounded font-medium">
          Exportar a Excel
        </button>
      </div>
    </div>
  </section>

  <section class="chart-section bg-white rounded-lg shadow-lg p-6">
    <div>
      <h2>Transacciones por Localidad</h2>
      <button onclick="exportMapaData()"
        class="bg-yellow-400 hover:bg-yellow-500 px-4 py-2 rounded font-medium">
        Exportar a Excel
      </button>
    </div>
  </div>

  <div>

    <section class="chart-section bg-white rounded-lg shadow-lg p-6">
      <div>
        <h2>Distribución por Área</h2>
        <button onclick="exportPieAreaData()"
          class="bg-yellow-400 hover:bg-yellow-500 px-3 py-1 rounded text-sm">
```

```

            Exportar
            </button>
        </div>
    </div>
</div>

</main>

<footer class="bg-gray-800 text-white text-center py-4">
    <p>&copy; 2025 Observatorio Técnico Catastral - UAEC</p>
</footer>

<script>
    // Datos centralizados
    const dataSets = {
        lineas: [], // datos para gráfico de líneas
        mapa: [], // datos para mapa
        pieArea: [], // datos para gráfico circular de área
        // ... más conjuntos de datos para los 15 gráficos
    };

    // Funciones de inicialización de gráficos
    function initLineasChart() {
        const chart = echarts.init(document.getElementById('chart-lineas'));
        chart.setOption({
            // configuración del gráfico
        });
    }

    // Funciones de exportación
    function exportLineasData() {
        const ws = XLSX.utils.json_to_sheet(dataSets.lineas);
        const wb = XLSX.utils.book_new();
        XLSX.utils.book_append_sheet(wb, ws, "Evolución");
        XLSX.writeFile(wb, "evolucion-transacciones.xlsx");
    }

    // Inicialización cuando el DOM está listo
    document.addEventListener('DOMContentLoaded', function() {
        initLineasChart();
        initMapaChart();
        // ... inicializar los otros 13 gráficos
    });
</script>
</body>
</html>

```

8.2 Organización Interna del Código

Secciones Claramente Definidas:

- **Head:** CDNs, estilos globales, configuración
- **Header:** Marca y navegación
- **Main Content:**
 - Gráficos principales (ancho completo)
 - Gráficos secundarios (cuadrícula de dos columnas)
- **Footer:** Información institucional
- **JavaScript:** Datos, funciones de gráficos, exportación

Patrones de Consistencia:

- **Estructura HTML:** Misma estructura para cada gráfico
- **Clases CSS:** Clases reutilizables y consistentes
- **Funciones JavaScript:** Convenciones de nomenclatura claras

- **Organización de datos:** Estructura uniforme de conjuntos de datos

8.3 Proceso de Modificación de Diseño

Para Cambios Comunes (con guía):

1. Para Cambios de Orden:
- Cortar y pegar secciones completas dentro del HTML
 - Mantener IDs únicos para evitar conflictos JavaScript
 - Verificar dependencias entre gráficos si existen
2. Para Cambios de Estilo:
- Modificar clases CSS globales
 - Actualizar variables de colores/fuentes centralizadas
 - Probar en diferentes dispositivos
3. Para Cambios de Contenido:
- Actualizar conjuntos de datos en sección JavaScript
 - Modificar configuraciones de gráficos
 - Verificar funciones de exportación

9. Comparación con Arquitectura Independiente

9.1 Tabla Comparativa Detallada

Aspecto	Gráficos Dependientes	Gráficos Independientes
Archivos a gestionar	✔ 1 archivo principal	✗ 15+ archivos HTML + JSON
Despliegue	✔ Una carga	✗ Carga coordinada múltiple
Respaldo/Restauración	✔ Un archivo	✗ 15+ archivos sincronizados
Depuración	✔ Un contexto	✗ Múltiples contextos
Flexibilidad de diseño	✗ Requiere código	✔ Plug-and-play Admin Web
Escalabilidad	✗ Archivo crece	✔ Agregar gráficos fácil
Mantenimiento de datos	⚠ Mixto	✔ JSON independientes
Rendimiento	✔ Una carga	⚠ Múltiples solicitudes
Consistencia visual	✔ CSS centralizado	⚠ Requiere coordinación
Curva de aprendizaje	✔ HTML tradicional	✗ Arquitectura modular
Aislamiento de fallos	✗ Un fallo afecta todo	✔ Fallos aislados
Productividad del equipo	✔ Un archivo enfoque	✗ Coordinación múltiple

9.2 Análisis de Casos de Uso

Caso 1: Actualización Regular de Datos

Gráficos Dependientes:

- Actualizar conjuntos de datos en sección JavaScript del archivo principal
- Una verificación de funcionamiento completo
- **Tiempo estimado:** 30 minutos

Gráficos Independientes:

- Actualizar 15 archivos JSON individuales
- Verificar funcionamiento de cada gráfico
- **Tiempo estimado:** 2 horas

Caso 2: Reorganización de Diseño**Gráficos Dependientes:**

- Modificar estructura HTML (requiere conocimientos técnicos)
- Probar funcionamiento después de cambios
- **Tiempo estimado:** 1-2 horas + conocimientos de código

Gráficos Independientes:

- Reorganizar inclusiones/iframes en página principal
- Sin modificación de código de gráficos
- **Tiempo estimado:** 15 minutos + conocimientos básicos Admin Web

Caso 3: Agregar Nuevo Gráfico**Gráficos Dependientes:**

- Agregar nueva sección HTML al archivo principal
- Agregar JavaScript de inicialización
- Integrar con conjuntos de datos existentes
- **Tiempo estimado:** 3-4 horas

Gráficos Independientes:

- Crear nuevo archivo HTML independiente
- Incluir en página principal
- **Tiempo estimado:** 2 horas

9.3 Decisión Estratégica: Cuándo Elegir Cada Arquitectura**Elegir Gráficos Dependientes cuando:**

- Equipo pequeño con conocimientos técnicos disponibles
- Diseño relativamente estable sin cambios frecuentes
- Prioridad en simplicidad de desarrollo y despliegue
- Recursos limitados para mantenimiento complejo
- Cronograma ajustado para entrega inicial

Elegir Gráficos Independientes cuando:

- Administrador Web sin conocimientos técnicos debe reorganizar contenido
- Diseño dinámico con cambios frecuentes esperados
- Equipo distribuido trabajando en gráficos específicos
- Escalabilidad importante con muchos gráficos futuros
- Flexibilidad es prioridad sobre simplicidad de desarrollo

10. Decisiones Relacionadas

10.1 ADRs que Aplican a Esta Arquitectura:

- **ADR002:** Selección de Apache ECharts (aplicable)
- **ADR003:** Implementación de funcionalidad de exportación a Excel (aplicable)
- **ADR004:** Uso de Tailwind CSS para estilos adaptativos (aplicable)
- **ADR005:** Uso de CDNs vs bibliotecas locales (aplicable)
- **ADR006:** Estructura JSON para actualización automática (parcialmente aplicable)

10.2 Decisiones Futuras Influenciadas:

- **Estrategia de pruebas:** Pruebas de un archivo vs. múltiples componentes
- **Estrategia de despliegue:** Despliegue de archivo único vs. despliegue orquestado
- **Gestión de datos:** Integrado vs. separado

11. Referencias

11.1 Arquitectura de Software

Atlassian. (2024). *Microservicios vs arquitectura monolítica: Comparación de metodologías de desarrollo*. <https://www.atlassian.com/microservices/microservices-architecture/microservices-vs-monolith>

Amazon Web Services. (2024). *La diferencia entre arquitectura monolítica y de microservicios*. <https://aws.amazon.com/compare/the-difference-between-monolithic-and-microservices-architecture/>

Coursera. (2024). *Microservicios vs arquitectura monolítica*. <https://www.coursera.org/articles/microservices-vs-monolithic-architecture>

11.2 Implementación Técnica

Apache Software Foundation. (2024). *ECharts - Documentación para integración de múltiples gráficos*. <https://echarts.apache.org/handbook/en/basics/import>

Tailwind Labs. (2024). *Tailwind CSS - Framework para estilos consistentes en aplicación monolítica*. <https://tailwindcss.com/docs>

World Wide Web Consortium. (2024). *HTML5 - Mejores prácticas para estructura de documento grande*. <https://www.w3.org/TR/html52/>

11.3 Casos de Estudio

GitHub Engineering. (2024). *Cómo manejar aplicaciones monolíticas grandes*. GitHub Engineering Blog.

Netflix Technology Blog. (2024). *Evolución de arquitectura: De monolítico a microservicios*. Netflix Tech Blog.

Shopify Engineering. (2024). *Monolito modular: Ejemplo de arquitectura híbrida exitosa*. Shopify Engineering Blog.

Documento controlado - Versión 1.0

- **Fecha de creación:** 8 de Octubre de 2025
- **Última modificación:** 8 de Octubre de 2025
- **Estado:** Aprobado
- **Próxima revisión programada:** Marzo 2026